作品介绍文档 • 2026-11-08

CrossExam 质询室

多 Agent 对抗式面试沙盘

参赛赛道：全国高校 Vibe Coding 开发挑战赛 · AI 创新应用挑战赛（赛道 10039）
作品形态：本地可运行的 Web 应用 + 免安装便携包 · 开源许可 MIT
代码仓库：https://github.com/yaoyuy/crossexam
文档日期：2026-11-08 · 对应仓库 main 分支最新提交

> 把你准备好的简历交给一整间面试官，先替你审一遍。 它不是让 AI 帮你写简历，是让一群 AI 面试官告诉你——简历里哪句话站不住。

本文件与评审要求的对照

平台要求	在本文的位置
(1) 描述项目详情及使用方式	一
(2) 描述项目的亮点	二
(3) AI 测试技术落地使用情况概述	三
(4) 描述项目中涉及的关键技术	四
(5) 项目相关的图片 / 照片	五
(6) 项目之前是否发表过或获奖	六
(7) 若基于其他项目优化，说明优化部分	七
附：交付物与访问链接	附录

一、项目详情及使用方式

1.1 它解决谁的什么问题

给谁用：正在投简历的求职者，尤其是应届生与转行者——他们不缺「把简历写好看」的工具，缺的是**在投出去之前，有人把简历里每一句话拷问一遍。**

什么问题：真正让人翻车的不是排版，而是**简历里写了、但本人其实答不上来的那句话。**面试官会顺着这句话往下追三层，而这三次追问，求职者在投递前从来没有预演过。

怎么解决：CrossExam 把这件事提前。喂进简历与目标岗位 JD，系统先抽出简历里一条条**可被追问的主张**（claims），再组一间面试官——技术、HR、业务主管、压力面试官、领域专家——并发追问。用户答完，系统给每条主张一个结论：站得住 / 表述过度 / 证据缺失 / 与原文矛盾 / 无法回答，并写明判定理由，最后**反推回简历**：该改哪一句、怎么改、改成什么。

一句话：**它不生产简历，它给简历做压力测试。**

1.2 一场质询的六个步骤

步骤	用户看到什么	系统在做什么
投料	简历与岗位 JD 两个输入框	从文件导入（PDF / Word / 文本）或粘贴，1–8 份 JD 均可
拆解	一列「可被追问的主张」	抽出主张并锚定简历原文的字符区间
审阅	岗位要求对照三组	逐份 JD 抽要求 → 跨份归并 → 与主张逐条对照：答得上 / 答得含糊 / 简历没写
组庭	三到五位面试官	按意图契约组庭，每人有目标、手法、禁区与预算
对抗	流式追问、可切慢放	多面试官并发追问，每一问都带简历原文锚点
判定	五类判定 + 理由	逐条落账，被拦下的提问单独记录
反推	改稿建议与并排对照	定位原文片段，给出改法；采纳后自动改简历并标绿

跑完一场可以带走三种报告（Markdown / 可打印 HTML / 真 .docx）、一份判定台账 Excel（五张表）、一个**无任何外链的单文件 HTML 分享包**，以及一版改完的简历（.txt / PDF / Word）。

1.3 三种使用方式

方式一：免安装便携包（评委零门槛） 解压 `crossexam-portable-0.1.0-win-x64.zip`（36 MB，自带 `node.exe`），双击 `启动.cmd` → 几秒后弹出一个没有地址栏的应用窗口。不写注册表、不装服务、不改 PATH，不用了删掉文件夹即可。

方式二：从源码（开发者与评审）

```
git clone https://github.com/yaoyuy/crossexam.git crossexam
cd crossexam && npm install && npm run preview
```

打开 http://127.0.0.1:8787 → 点「一键示例」→ 点「开始质询」。**不需要 API Key，不需要数据库，不需要 Docker。**

方式三：接真实模型 在页面里把运行模式切到「联网直连」即可（不必重启服务）；填 `.env` 里的 `LLM_BASE_URL / LLM_API_KEY / LLM_MODEL`，任何 OpenAI 兼容端点都行。也可以切「联网录制」，把这一场录成新卡带。

1.4 为什么不需要 API Key

默认运行在**回放模式**：仓库里带着 149 盘已录制的模型响应（`fixtures/llm/*.json`），按**提示词指纹**寻址。因此任何一台机器、任何一个时间点，跑出来的都是同一场质询——这既是评委不用配任何东西就能验收的原因，也是本项目全部自动化测试的技术底座（详见第三节）。

二、项目亮点

亮点 1：提问没有原文锚点，界面直接不显示。 每条提问必须携带简历原文的字符区间；模型给了提问却没给锚点，前端拒显。这不是提示词里写「请不要编造」，而是把编造提问的结构性出口焊死。

亮点 2：追问次数在代码里记账，不是在人设里许愿。 每主张轮数、每位面试官额度、全场总量三处同时记账（温和 / 常规 / 挑刺 = 每主张 2/3/4 轮、全场 40/60/80 次）。prompt 里写「最多问 3 次」模型可以不听；记账闸门让死循环在结构上不成立。

亮点 3：面试官人设是结构化契约，能摊开看、能对账。 每位面试官的人设、目标、手法、禁区、预算是一份 JSON（意图契约），可版本化、可 diff；界面左栏能把契约与他实际拿到的 System Prompt 一起展开，并与运行时指纹对账。

亮点 4：同一输入永远同一结果，离线可跑完整场。 模型调用按提示词指纹落盘成卡带，回放模式下整场质询 ≈0.1 秒跑完（预算 8000 ms）。改动提示词就必须重录——这条约束让「测试通过」不会因为模型随机性而变成运气。

亮点 5：判定要拿得出证据，被拦下的提问也留档。 五类判定每条都带理由与原文引用；模型试图越过锚点、越过预算、越过 schema 的提问被拦下，单独记入「被拦下的」，可导出查看。**不静默降级，缺哪盘卡带就响亮报错。**

亮点 6：连「过程」本身都可复现。 400 余次提交，每一次都对应一次可独立说明的改动、30 篇架构决策记录（ADR）、64 份 AI 协同 Prompt 链快照（其中 58 份为分页分片，26.7 MB）、五道提交门禁、干净目录复现实测 **99.3 秒**（上限 600 秒）。

三、AI 测试技术落地使用情况概述

3.1 一句话定位

本项目把「AI 输出到底对不对」这件事，从**看过就算**变成**可执行、可复跑、可举证**的测试：既用 AI 去做测试，也用测试去管住 AI。

3.2 三个层次的落地

第一层：把 AI 输出当作被测系统（SUT） 五个面试官角色就是五个测试角色，每条主张是一条测试用例，每条判定是一份测试结论，每条「被拦下的」是一条缺陷记录。测试对象不是代码函数，而是「模型在给定契约与预算下会不会越界」——锚点越界、预算越界、schema 越界三类，全部有独立断言。

第二层：用工程手段收住生成式不确定性（四个机制，缺一不可）

机制	做法	拦住了什么
录制回放（record / replay）	模型响应按提示词指纹落盘成 149 盘卡带	让 AI 测试从「跑一次看运气」变成确定性回归
输出契约（zod）	所有模型输出先过 schema 再入库	自由文本直进数据库、字段缺失导致的半截状态
锚点复定位	判定前用程序重新定位引文区间	模型自述「我引用了原文」但实际引错、引空
预算记账	轮数 / 额度 / 总量三处计数，超支即拒	追问死循环、token 失控、整场跑不完

补充一条同类的：**判定结论由代码改写，而不是照抄模型措辞**——模型可以建议，落库的字段由程序重算，避免「模型自己宣布自己通过」。

第三层：测试工程化落到提交门禁

门禁	规模	拦住了什么
类型检查	三个包全量	接口漂移
单元与集成测试	73 个测试文件 / **767 条断言**	确定性内核（锚定、预算、契约、收敛）的行为回归
真进程冒烟	9 项	同进程 `app.request` 看不见的坑：表没建、端口没监听、SSE 不结束
构建	56 modules / CSS 48.26 kB / JS 416.59 kB（gzip 125.0 kB）	前端产物可生成、体积不失控
性能预算	14 项	起服 422 ms / 整场回放 90 ms / 报告 7750 字符 / 打包 22 KB 等上界
界面点测（另跑）	187 项真浏览器断言 + 36 张截图，控制台 0 报错	「接口是对的、界面没接上」这一类

3.3 这类测试实际抓到过的问题（举三例）

1. **界面点测抓到「面板顺序跟着点击历史走」**：单测与冒烟都看不见——接口返回的字段全对，错的是 DOM 顺序。187 项真浏览器断言第一次跑就抓到了。
2. **界面点测抓到「连点删除打到已删会话」**：竞态问题，只有在真浏览器里连点才暴露；后来补了幂等断言。
3. **真进程冒烟抓到「表没建」**：单元测试用的是同进程 `app.request`，看不到进程边界上的初始化顺序；冒烟第一次跑就红了。

教训是同一句话：**不同层级的门禁拦的是不同类别的错，任何一层都替代不了另一层。**

3.4 实测数字（全部来自本仓库真实命令输出）

指标	实测值	出处
提交门禁	五道全绿	`npm run verify`
测试	73 文件 / 767 条	同上
真进程冒烟	9 项	同上
性能预算	14 项，起服 ≈430 ms（预算 5000）、整场回放 ≈100 ms（预算 8000）	同上
界面点测	187 项，控制台 0 报错	`npm run e2e:ui`
界面截图	36 张，sha256 去重 0 重复	`npm run shots`
干净目录复现	99.3 秒（clone 24.1 s / install 31.1 s / 门禁 36.7 s / 跑一场 1.1 s）	`npm run repro`
离线卡带	149 盘	`fixtures/llm/`

3.5 局限（如实说明）

- 149 盘卡带中有一部分是**合成演示数据**（`model` 字段为 `scripted-demo`），不是真实模型输出，目的是让评委离线可跑；接真实模型的路径是「联网直连 / 联网录制」，但**未做真实模型下的压力与长稳测试。**
- 界面点测依赖本机 Edge / Chrome，因此不进 `npm run verify`（避免没有浏览器的机器上误红），作为独立门禁另跑。
- 判定质量目前只有规则与一致性测试守着，**没有人工标注集做准确率评估**，这是本作品下一步最该补的测试缺口。

四、项目中涉及的关键技术

4.1 四个技术支点

支点	一句话	为什么不是装饰
证据锚定	每条提问必须携带简历原文的字符区间	提问没有锚点前端拒显，从设计上排除幻觉式提问
追问预算	每主张轮数 / 每人额度 / 全场总量在代码里记账	prompt 写「最多问 3 次」模型可以不听，记账闸门让死循环不成立
意图契约	每个 Agent 的人设、目标、手法、禁区、预算是结构化 JSON	可解释、可版本化、可 diff，界面能摊开并与运行时指纹对账
可复现回放	每次模型调用按提示词指纹落盘成卡带	无 Key 离线跑通全流程，同一输入永远同一结果

4.2 技术栈与结构

- **运行时**：Node 24 + TypeScript，**服务端零构建**——Node 原生剥离类型直接跑 `.ts`（ADR-0001），少一层产物就少一处「忘了 build」。
- **服务端**：Hono + `@hono/node-server`，SSE 直播九阶段流水线（五个核心阶段 + 读 JD 三次 + 收尾一次反问）；存储用 Node 内置 `node:sqlite`（单文件，零原生依赖）。
- **前端**：Vite 8 + React 19 + Tailwind v4，五栏界面（投料台 / 质询舞台 / 岗位要求对照 / 简历原文与 JD 原文分屏联动高亮）。
- **约束收敛**：zod 作为模型输出的唯一入口。
- **测试**：vitest + 自研真进程冒烟 + 自研 CDP 界面点测。

```
packages/shared/    确定性内核：领域模型、锚定、切分、预算、契约、zod 收敛（纯函数，可单测）
packages/server/    Hono + SSE：九阶段流水线、LLM 适配层（live / record / replay）、node:sqlite
packages/web/       Vite + React 五栏界面
tools/              导出 Prompt 链 / 冒烟 / 截图 / 复现预演 / 性能预算 / 便携包 / 文档构建
fixtures/llm/       149 盘模型响应卡带（离线回放数据）
docs/               ADR 30 篇、Prompt 链快照、截图、技术文档源与 PDF、视频脚本
```

4.3 三条工程约束（自设红线）

1. **禁止任何需要本地编译的原生依赖**——否则「十分钟复现」会碎在评委机器上。
2. **模型输出必须过 zod**，不许自由文本直接入库。
3. **不允许无锚点的提问**——没有绑定简历原文区间的提问不许进界面。

4.4 隐私

简历属于个人隐私数据：只落本机 SQLite，不上传任何第三方；仓库与 Prompt 链快照做过脱敏（姓名、手机号、邮箱、院校逐字替换，历史提交亦已重写）。

五、项目相关的图片 / 照片

以下截图全部由 `npm run shots` 用真浏览器（Edge / Chrome + CDP）自动生成，不是手工拼图；控制台有任何报错即失败。

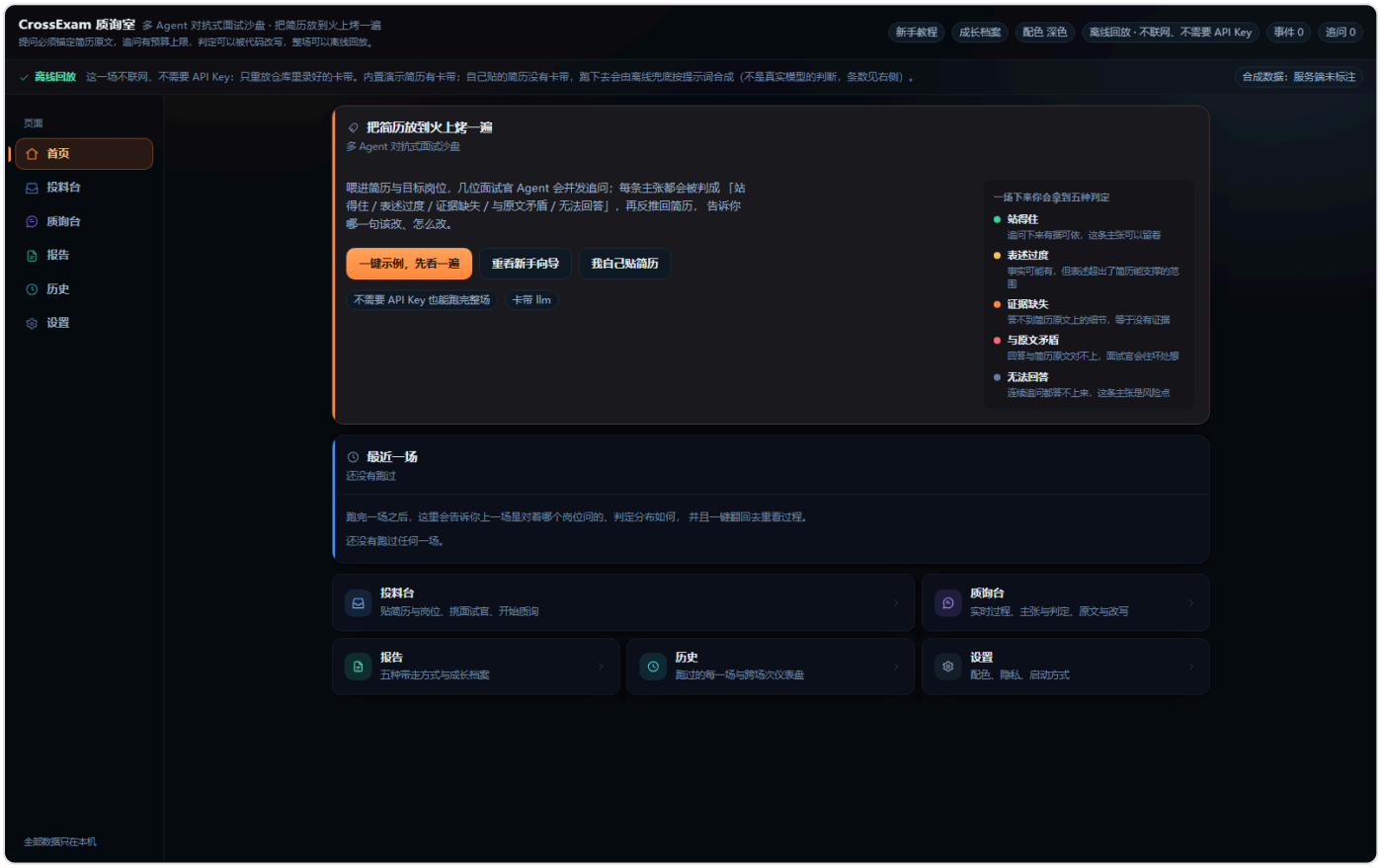


图 1 · 首页。左栏投料（简历 + 岗位 JD），中栏质询舞台，右栏简历原文与 JD 原文分屏联动高亮。

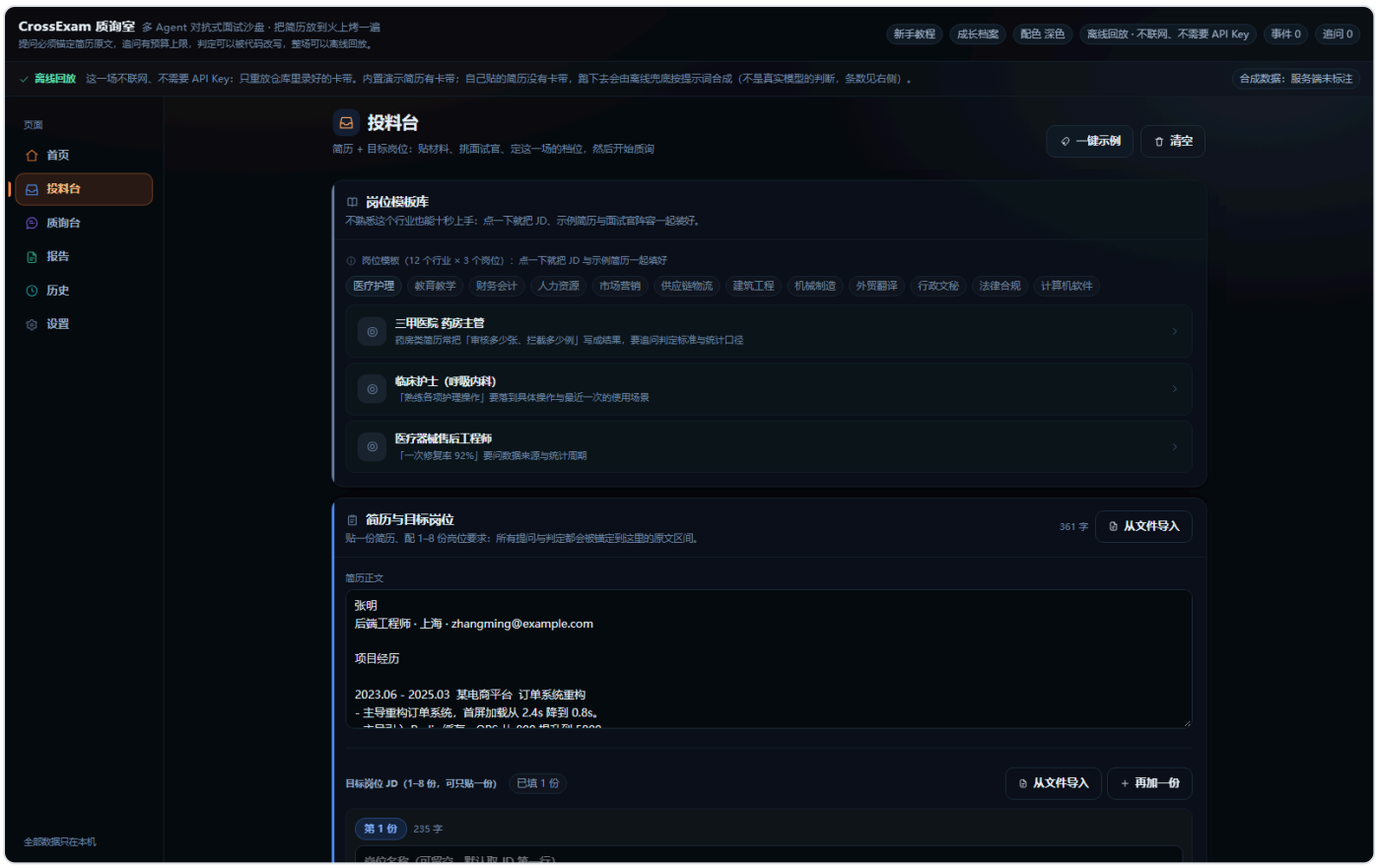


图 2 · 点「一键示例」后，模板岗位、示例简历与卡带一起就位，评委不用准备任何文件就能开跑。

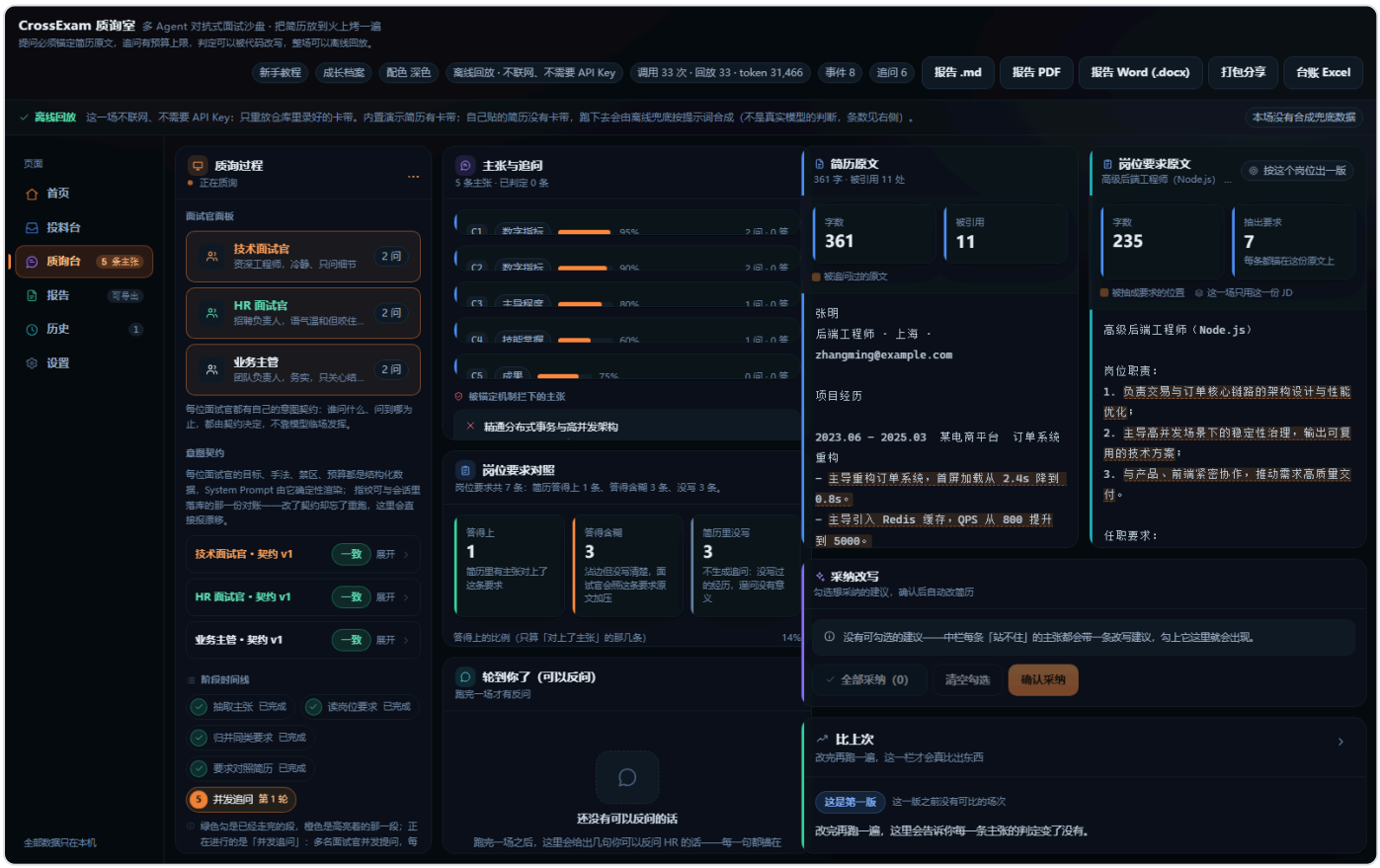


图 3 · 对抗阶段。每次追问同时在简历与 JD 上高亮原文片段，点任一处右栏就聚焦到那一段。

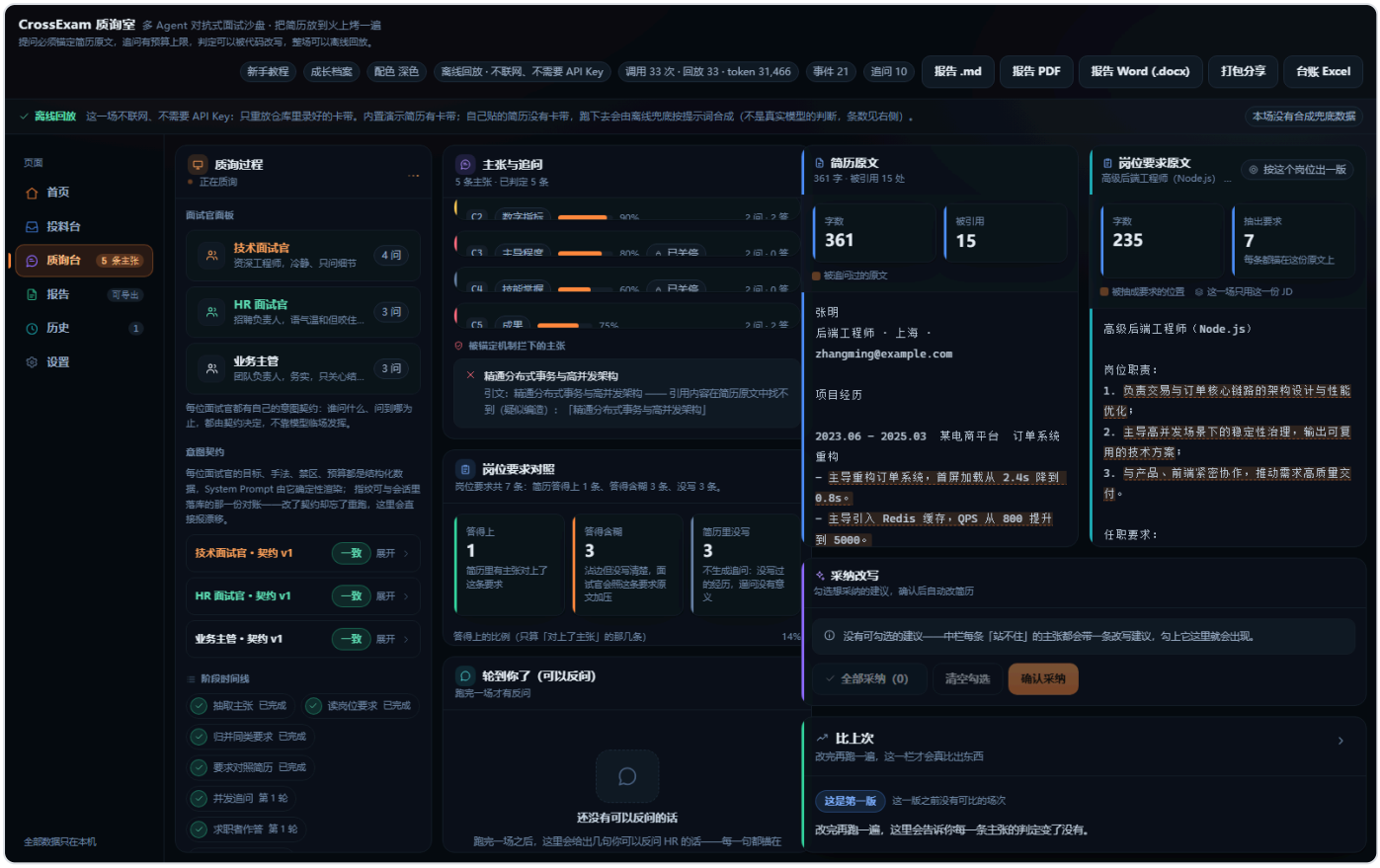


图 4 · 判定。五类结论逐条落账，每条写明为什么这样判、引了哪段原文。



CrossExam 质询报告

多 Agent 对抗式面试沙盘 · 提问锚定简历原文，追问有预算，判定可被代码改写

会话	s_ec5f26c6（2026-11-08T14:21:40.111Z）
目标岗位	高级后端工程师（Node.js） · 演示简历
简历指纹	e3ddedd7a4a16f1e6d42e297
跑法	replay，模型调用 21 次（其中离线回放 21 次）
Token	输入 19415 / 输出 3199 / 缓存命中 0 / 推理 259
追问强度	常规（每主张 3 轮 · 每人每主张 2 次 · 全场 60 次）
改写风格	常规
追问预算	已用 5 次，上限 60 次；主张 5 条
岗位要求	1 份 JD，抽出 7 条要求（答得上 1 · 答得含糊 3 · 简历里没写 3）
反问	4 句（每句都带简历或 JD 原文依据）

零、过程说明（如实记录，不掩饰）

- 有 1 条主张因为锚不到简历原文被丢弃
- 第 2 轮没有任何新的追问，质询到此结束

比上次

上一版：s_aac8db7a（2026-11-08T14:21:25.662Z，高级后端工程师（Node.js） · 演示简历）

站得住 1 → 5 条（共 5 → 5 条判定），变好 4 条，追问 10 → 5 次

- 简历内容：改过了（两份的指纹不同）

变好的（4 条）

- C2 · 引入 Redis 缓存，把接口 QPS 从 800 提升到 5000：表述过度 → 站得住（变好）
- C3 · 独立完成 RBAC 权限模块，覆盖 12 个业务线：证据缺失 → 站得住（变好）
- C4 · 学习能力强、抗压能力强：无法回答 → 站得住（变好）
- C5 · 参与的项目获得公司年度技术突破奖：证据缺失 → 站得住（变好）

一、主张与判定

C1 站得住 数字指标 · 权重 0.95

主导订单系统重构，把首屏加载从 2.4s 优化到 0.8s

图 5 · 报告页。Markdown / 可打印 HTML / 真 .docx 三种形态，改完的简历另三种。



图 6 · 浅色主题。深色 / 浅色两套配色，头部一键切换并记住选择。

六、项目之前是否发表过或获奖

如实说明：没有。

- **未发表**：没有投过期刊、会议或任何论文/专利，本项目是纯工程作品。
- **未获奖**：截至 2026-11-08，本作品尚未参加过任何评奖，本次为**首次参赛**。
- **已公开**：代码以 MIT 许可开源在 GitHub（https://github.com/yaoyuy/crossexam），仓库 400 余次提交、CI 全绿；社区无 fork 或引用记录可查。
- **可核查的过程证据**：AI 协同 Prompt 链快照 64 份（26.7 MB）、架构决策记录 30 篇（docs/adr/）、踩坑与发现 100 余条（findings.md）、逐次会话开发日志（progress.md）。

七、是否基于其他项目优化

不是基于任何现有项目的二次开发，是从空目录自建。

- **无 fork、无换皮**：仓库首次提交即本项目自身代码，无上游仓库、无 fork 关系；400 余次提交全部对应本项目自己的功能迭代（赛题明确禁止 fork 他人项目换皮参赛）。
- **仅使用通用开源库作为依赖**，全部为宽松许可证，保留版权声明：

依赖	许可证	用途
Hono / @hono/node-server	MIT	HTTP 与 SSE
React / Vite / Tailwind CSS	MIT	界面与构建
zod	MIT	模型输出 schema
vitest	MIT	测试
unpdf	MIT	PDF 文本解析

- **不含 GPL / AGPL / 无许可证代码**；没有任何需要本地编译的原生依赖（这是「十分钟复现」的前提）。

- **创意层面的参考**：调研阶段阅读过若干公开的开源实现（会话回放、Agent 编排、结构化契约），只借鉴思路并自行重写，不复制代码；取材红线写在 docs/roadmap-v2.md 第 2 节。
- **数据层面的原创**：岗位模板 JD 由本项目自行撰写，**未抓取任何招聘平台数据**；用户自己的 JD 由用户自行上传。

附录：交付物与访问链接

交付物	位置
公开代码仓库	https://github.com/yaoyuy/crossexam （MIT，397 提交）
演示视频	仓库内 docs/video/crossexam-demo.mp4（6 分 22 秒 / 1080p / 41.6 MB，AI 协同段 2 分 13 秒）
技术文档	仓库内 docs/CrossExam-技术文档.pdf（30 页）
AI 协同 Prompt 链	仓库内 docs/prompt-chain/（64 份，26.7 MB）
免安装便携包	Release v0.1.0 附件（36 MB，Windows x64）
完整文件包	平台第 5 项网盘链接（文档 + 视频 + 源码 + Prompt 链 + 便携版）